

미니 단어장 v1 — 구조 해설 & 코드 입력 자료

이 자료는 강의 2~3주차(기본 클래스/객체, 배열, 반복문, 예외처리)에 맞춘 콘솔 단어장 v1의 구조를 먼저 설명하고, 뒤에 전체 코드를 실어 학생이 직접 타이핑하며 연습할 수 있도록 만들었습니다.

1. 무엇을 만드는가

터미널에서 실행되는 영단어장 프로그램입니다. 메뉴 번호를 입력하면 단어를 **추가·조회·검색·삭제**할 수 있고, 등록된 단어는 프로그램을 종료하면 사라집니다 (파일이나 DB에 저장하지 않음 — 저장은 이후 버전(v3)에서 다룹니다).

- | | |
|----------|-----------------|
| 1. 단어 추가 | 2. 전체 목록 (알파벳순) |
| 3. 단어 검색 | 4. 단어 삭제 |
| 0. 종료 | |

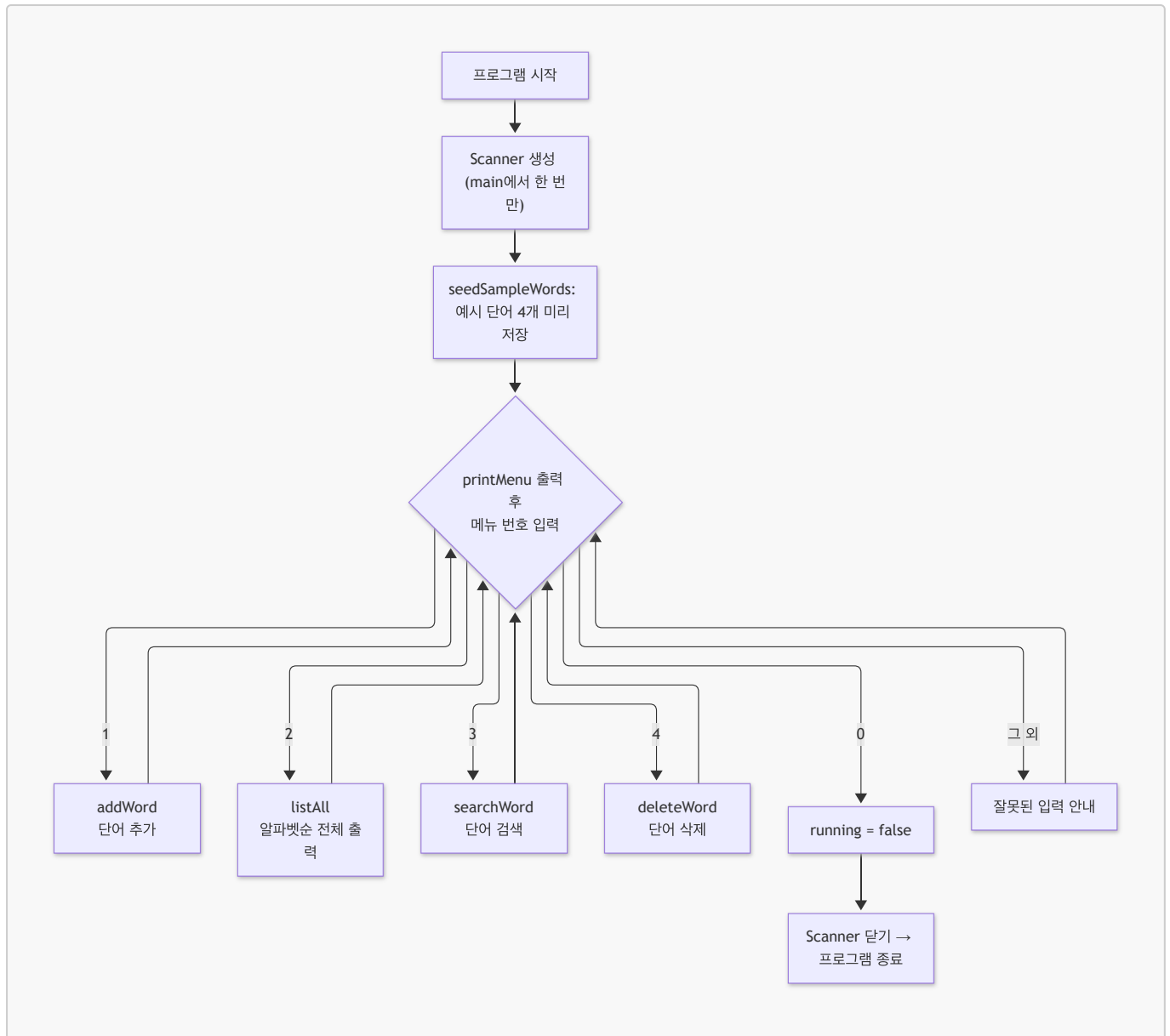
2. 코드 구조 — 왜 파일이 2개뿐인가

파일	역할
<code>Word.java</code>	단어 하나(영단어/뜻/예문/난이도)를 표현하는 클래스 — 데이터 보관 담당
<code>VocabAppV1.java</code>	<code>main()</code> 과 메뉴 기능 — 동작(로직) 담당

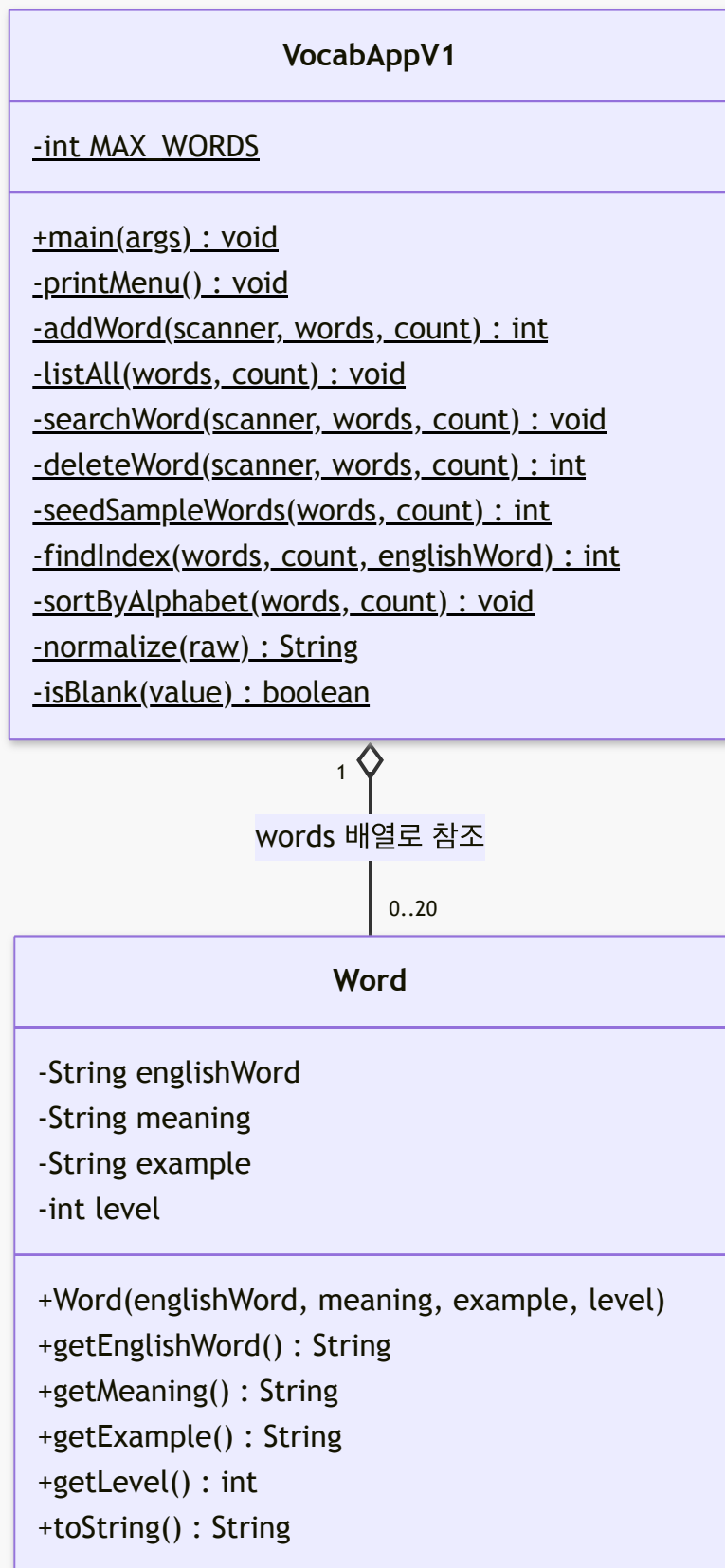
아직 패키지, 상속, 컬렉션을 배우기 전이라 파일을 더 잘게 나눌 이유가 없습니다. "데이터(Word)"와 "그 데이터를 다루는 동작(VocabAppV1)"만 분리한, 가장 단순한 클래스 설계입니다.

설계 규칙 하나: `VocabAppV1`의 모든 메서드는 `Scanner, Word[] words, int count`를 매개변수로 주고받습니다. 아직 인스턴스 필드(객체가 자기 상태를 들고 있는 방식)를 배우기 전이라, "값은 항상 매개변수로 전달한다"는 원칙 하나로 통일했습니다. 특히 `Scanner(System.in)`은 프로그램 전체에서 하나만 만들어 `main()`에서 딱 한 번만 닫습니다 — 여러 개 만들면 입력 버퍼가 꼬이고, 하나를 닫으면 `System.in` 자체가 닫혀서 다른 `Scanner`도 못 쓰게 되기 때문입니다.

3. 프로그램 흐름도



4. 클래스 구조



\$ 표시는 **static** 메서드/필드라는 뜻입니다. **VocabAppV1**은 객체를 만들지 않고 **main()**부터 모든 메서드를 **static**으로 바로 호출합니다.

5. 주요 멤버 설명

Word — 단어 하나를 표현하는 클래스

멤버	설명
<code>englishWord, meaning, example, level</code>	생성자에서 한 번 정해지면 바뀌지 않는(final) 필드
<code>Word(...)</code> 생성자	문자열은 공백 제거·소문자 변환 후 저장
<code>getEnglishWord()</code> 등 getter	외부에서 필드 값을 읽는 유일한 방법 (필드는 private)
<code>toString()</code>	난이도를 "초급/중급/고급" 한글로 바꿔 "[초급] apple - 사과 (예: ...)" 형태로 출력

VocabAppV1 — 실행과 메뉴 로직

메서드	설명
<code>main</code>	Scanner·배열 생성, 예시 단어 등록, 메뉴 반복(while) 실행
<code>printMenu</code>	메뉴 화면 출력
<code>addWord</code>	새 단어 입력 → 빈 값/중복 검사 → 난이도 입력(예외처리) → 배열에 저장
<code>listAll</code>	<code>sortByAlphabet</code> 로 정렬 후 번호를 붙여 출력
<code>searchWord</code>	<code>findIndex</code> 로 찾은 단어를 출력, 없으면 안내 메시지
<code>deleteWord</code>	찾은 위치부터 뒤 요소를 한 칸씩 당겨서 삭제
<code>seedSampleWords</code>	시작할 때 예시 단어 4개를 배열에 채움
<code>findIndex</code>	영어 단어(정규화 후)가 일치하는 배열 위치를 선형 탐색
<code>sortByAlphabet</code>	선택 정렬(selection sort)로 알파벳순 정렬
<code>normalize</code>	비교 전에 공백 제거 + 소문자 변환
<code>isBlank</code>	문자열이 null 이거나 공백뿐인지 확인

예외처리가 등장하는 곳

```
try {
    level = Integer.parseInt(levelChoice);
} catch (NumberFormatException e) {
    System.out.println("숫자가 아니라서 1) 초급으로 저장합니다.");
    level = 1;
}
```

숫자인지 미리 검사하는 우회 코드 대신, "일단 시도해보고 실패하면 잡는다"는 예외처리 본연의 방식을 `addWord()`에서 처음 연습합니다.

6. 코드 입력 안내

아래 두 파일의 이름과 코드를 순서대로 그대로 입력하세요. 위에서 이미 구조와 동작을 설명했기 때문에 코드에는 줄별 주석을 넣지 않았습니다.

Word.java

```
public class Word {

    private final String englishWord;
    private final String meaning;
    private final String example;
    private final int level;

    public Word(String englishWord, String meaning, String example, int
level) {
        this.englishWord = englishWord == null ? "" :
englishWord.trim().toLowerCase();
        this.meaning = meaning == null ? "" : meaning.trim();
        this.example = example == null ? "" : example.trim();
        this.level = level;
    }

    public String getEnglishWord() {
        return englishWord;
    }

    public String getMeaning() {
        return meaning;
    }

    public String getExample() {
        return example;
    }

    public int getLevel() {
        return level;
    }

    @Override
    public String toString() {
        String levelLabel;
        if (level == 2) {
            levelLabel = "중급";
        } else if (level == 3) {
            levelLabel = "고급";
        } else {
            levelLabel = "초급";
        }

        String base = "[" + levelLabel + "] " + englishWord + " - " +
meaning;
        if (!example.isEmpty()) {
            return base + " (예: " + example + ")";
        }
        return base;
    }
}
```

```
}  
}
```

VocabAppV1.java

```
import java.util.Scanner;

public class VocabAppV1 {

    private static final int MAX_WORDS = 20;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Word[] words = new Word[MAX_WORDS];
        int count = 0;
        count = seedSampleWords(words, count);

        boolean running = true;
        while (running) {
            printMenu();
            System.out.print("선택: ");
            String choice = scanner.nextLine();

            switch (choice) {
                case "1":
                    count = addWord(scanner, words, count);
                    break;
                case "2":
                    listAll(words, count);
                    break;
                case "3":
                    searchWord(scanner, words, count);
                    break;
                case "4":
                    count = deleteWord(scanner, words, count);
                    break;
                case "0":
                    running = false;
                    break;
                default:
                    System.out.println("잘못된 입력입니다. 메뉴 번호를 다시 선택하세
요.");
            }

            System.out.println("프로그램을 종료합니다. (등록된 단어 " + count + "개는 저
장되지 않습니다)");
            scanner.close();
        }

        private static void printMenu() {
            System.out.println();
            System.out.println("==== 미니 단어장 (v1) =====");
            System.out.println("1. 단어 추가");
        }
    }
}
```

```
        System.out.println("2. 전체 목록 (알파벳순)");
        System.out.println("3. 단어 검색");
        System.out.println("4. 단어 삭제");
        System.out.println("0. 종료");
    }

    private static int addWord(Scanner scanner, Word[] words, int count) {
        if (count >= words.length) {
            System.out.println("더 이상 추가할 수 없습니다 (최대 " + words.length
+ "개, 배열이 꽉 찼습니다).");
            return count;
        }

        System.out.print("영어 단어: ");
        String englishWord = scanner.nextLine();
        if (isBlank(englishWord)) {
            System.out.println("단어를 입력하지 않아 취소합니다.");
            return count;
        }
        if (findIndex(words, count, englishWord) != -1) {
            System.out.println("이미 등록된 단어입니다.");
            return count;
        }

        System.out.print("뜻: ");
        String meaning = scanner.nextLine();
        System.out.print("예문 (없으면 그냥 엔터): ");
        String example = scanner.nextLine();

        System.out.println("난이도를 선택하세요. 1) 초급 2) 중급 3) 고급");
        System.out.print("선택: ");
        String levelChoice = scanner.nextLine();

        int level;
        try {
            level = Integer.parseInt(levelChoice);
            if (level < 1 || level > 3) {
                System.out.println("1~3 범위가 아니어서 1) 초급으로 저장합니다.");
                level = 1;
            }
        } catch (NumberFormatException e) {
            System.out.println("숫자가 아니어서 1) 초급으로 저장합니다.");
            level = 1;
        }

        Word word = new Word(englishWord, meaning, example, level);
        words[count] = word;
        count++;
        System.out.println("저장했습니다: " + word);
        return count;
    }

    private static void listAll(Word[] words, int count) {
        if (count == 0) {
```

```
        System.out.println("등록된 단어가 없습니다.");
        return;
    }
    sortByAlphabet(words, count);
    for (int i = 0; i < count; i++) {
        System.out.println((i + 1) + ". " + words[i]);
    }
}

private static void searchWord(Scanner scanner, Word[] words, int
count) {
    System.out.print("검색할 단어: ");
    String key = scanner.nextLine();
    int index = findIndex(words, count, key);
    System.out.println(index != -1 ? words[index].toString() : "등록되지
않은 단어입니다.");
}

private static int deleteWord(Scanner scanner, Word[] words, int count)
{
    System.out.print("삭제할 단어: ");
    String key = scanner.nextLine();
    int index = findIndex(words, count, key);
    if (index == -1) {
        System.out.println("등록되지 않은 단어입니다.");
        return count;
    }

    for (int i = index; i < count - 1; i++) {
        words[i] = words[i + 1];
    }
    words[count - 1] = null;
    System.out.println("삭제했습니다.");
    return count - 1;
}

private static int seedSampleWords(Word[] words, int count) {
    words[count++] = new Word("apple", "사과", "I eat an apple every
day.", 1);
    words[count++] = new Word("book", "책", "This book is
interesting.", 1);
    words[count++] = new Word("achieve", "성취하다", "She achieved her
goal.", 2);
    words[count++] = new Word("ambiguous", "애매한", "The instructions
were ambiguous.", 3);
    return count;
}

private static int findIndex(Word[] words, int count, String
englishWord) {
    String key = normalize(englishWord);
    for (int i = 0; i < count; i++) {
        if (words[i].getEnglishWord().equals(key)) {
            return i;
        }
    }
}
```

```
    }
    }
    return -1;
}

private static void sortByAlphabet(Word[] words, int count) {
    for (int i = 0; i < count - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < count; j++) {
            if
(words[j].getEnglishWord().compareTo(words[minIndex].getEnglishWord()) < 0)
{
                minIndex = j;
            }
        }
        if (minIndex != i) {
            Word temp = words[i];
            words[i] = words[minIndex];
            words[minIndex] = temp;
        }
    }
}

private static String normalize(String raw) {
    return raw == null ? "" : raw.trim().toLowerCase();
}

private static boolean isBlank(String value) {
    return value == null || value.trim().isEmpty();
}
}
```